

Creating image pull secret for IBM Cloud® based installatic

Last Updated: 2025-09-04

With OpenShift® Container Platform, you can create a global image pull secret that each worker node in the cluster can use to pull images from a private registry.

Before you begin [🔗](#)

- Download the jq JSON processor command line package. To download the jq JSON processor package, see [Download the jq JSON processor command line package](#).
- Ensure that you have access to https://cloud.ibm.com/docs/openshift?topic=openshift-access_cluster.

About this task [🔗](#)

To add private registries, edit the global pull-secret in the openshift-config project. You can set up your Red Hat OpenShift on IBM Cloud cluster.

! **Important:** Do not replace the global pull secret with a pull secret that does not have credentials to the default Red Hat registries. If you do, the cluster will not be able to pull images from the default registries. You must only update the global pull secret to add more auths and never overwrite.

Procedure [🔗](#)

1. To add private registries, edit the global pull-secret in the openshift-config project as follows:
 - a. Create a secret value that holds the credentials to access your private registry and store the decoded secret value in a JSON file. When you create the secret, the decoded secret value is then stored in a JSON file to later use in your global pull secret.

```
oc create secret docker-registry <secret_name> --docker-server=<registry_URL> --docker-username=<docker_username> --docker-password=<docker_password>
```

The following parameters are mandatory.

- **--namespace <project>**

The Red Hat OpenShift project of your cluster where you want to use the secret and deploy containers to. To list available projects in your cluster, run the following command:

```
oc get ns
```

- **<secret_name>**

The name that you want to use for your image pull secret.

- **--docker-server <registry_URL>**

The URL to the registry where your private images are stored.

- **--docker-username <docker_username>**

The username to log in to your private registry.

- **--docker-password <token_value>**

The password to log in to your private registry, such as a token value.

- **--docker-email <docker-email>**

If you have Docker account, enter your docker email address. If you don't have one, enter a fictional email address, such as a@b.c. This email address is used to send you notifications about your account.

The following parameters are not mandatory.

- **--dry-run=true**

Include this flag to create the secret value only, and not create and store the secret object in your cluster.

- **--output="jsonpath={.data.\.dockerconfigjson}"**

Get only the .dockerconfigjson value from the data section of the Kubernetes secret.

- **| base64 --decode > myregistryconfigjson**

Download the decoded secret data to a local myregistryconfigjson file.

- b. Retrieve the decoded secret value of the default global pull secret and store the value in a dockerconfigjson file.

```
oc get secret pull-secret -n openshift-config --output="jsonpath={.data.\.dockerconfigjson}" | base64 --decode > dockerconfigjson
```

- c. Combine the downloaded private registry pull secret myregistryconfigjson file with the default global pull secret dockerconfigjson file

```
jq -s '.[0] * .[1]' dockerconfigjson myregistryconfigjson >
dockerconfigjson-merged
```

- d. Update the global pull secret with the combined dockerconfigjson-merged file.

```
oc set data secret/pull-secret -n openshift-config --from
-file=.dockerconfigjson=dockerconfigjson-merged
```

- e. Verify that the global pull secret is updated. Check that your private registry and each of the default Red Hat registries are in the output of the following command:

```
oc get secret pull-secret -n openshift-config
--output=jsonpath={.data.\.dockerconfigjson} | base64
--decode
```

Sample output

```
{
  "auths": {
    "cloud.openshift.com": {
      "auth": "<encoded_string>",
      "email": "email@example.com"
    },
    "quay.io": {
      "auth": "<encoded_string>",
      "email": "email@example.com"
    },
    "registry.connect.redhat.com": {
      "auth": "<encoded_string>",
      "email": "email@example.com"
    },
    "registry.redhat.io": {
      "auth": "<encoded_string>",
      "email": "email@example.com"
    },
    "<private_registry>": {
      "username": "iamapikey",
      "password": "<encoded_string>",
      "email": "email@example.com",
      "auth": "<encoded_string>"
    }
  }
}
```

- f. To pick up the global configuration changes, reload all the worker nodes in your cluster.

- i. Note the **ID** of the worker nodes in your cluster.

```
ibmcloud oc worker ls -c <cluster_name_or_ID>
```

- ii. Reload each worker node. You can reload multiple worker nodes by including multiple -w flags, but make sure to keep enough worker node

Note: For IBM VPC type clusters, the **Reload** option is not available, instead you can use the **Replace** option.

```
ibmcloud oc worker reload -c <cluster_name_or_ID>
-w <workerID_1> -w <workerID_2>
```

- g. After the worker nodes are back in a healthy state, verify that the global pull secret is updated on a worker node.

- i. Start a debugging pod to log in to a worker node. Use the **Private IP** that you retrieved earlier for the <node_name>.

```
oc debug node/<node_name>
```

- ii. Change the root directory to the host so that you can view files on the worker node.

```
chroot /host
```

- iii. Verify that the Docker configuration file has the registry credentials that match the global pull secret that you set.

```
vi /.docker/config.json
```

2. Setting up a cluster to pull entitled software as follows:

Entitled software is stored in a special IBM Cloud Container Registry `cp.icr.io` domain. To access this domain, you must create an image pull secret for the software.

- a. Get the entitlement key for your entitled software library.
 - i. Log in to [MyIBM.com](https://myibm.com) and scroll to the **Container software library** section. Click **View library**.
 - ii. From the **Access your container software**, **Entitlement keys** page, click **Copy key**. This key authorizes access to all the entitled software in the library.
- b. In the project that you want to deploy your entitled containers, create an image pull secret so that you can access the `cp.icr.io` entitled registry [private registries](#).

```
oc create secret docker-registry entitled-cp-icr-io --docker-server=cp.icr.io --docker-username=cp --docker-password=<entitlement_key>
```

- c. Add the image pull secret to the service account of the namespace so that any container in the project can use the entitlement key to pull entitled software.

```
oc patch -n <project> serviceaccount/default --type='json' -p='[{"op": "add", "path": "/imagePullSecrets/-", "value": {"name": "entitled-cp-icr-io"}}']'
```

- d. Create a pod in the project that builds a container from an image in the entitled registry.

```
oc run <pod_name> --image=cp.icr.io/<image_name> -n <project> --generator=run-pod/v1
```

- e. Check that your container was able to successfully build from the entitled image by verifying that the pod is in a **Running** status.

```
oc get pod <pod_name> -n <project>
```

- f. To pick up the global configuration changes, reload all the worker nodes in your cluster.

- i. Note the **ID** of the worker nodes in your cluster.

```
ibmcloud oc worker ls -c <cluster_name_or_ID>
```

- ii. Reload each worker node. You can reload multiple worker nodes by including multiple `-w` flags, but make sure to keep enough worker nodes to maintain the cluster.



Note: For IBM VPC type clusters, the **Reload** option is not available, instead you can use the **Replace** option.

```
ibmcloud oc worker reload -c <cluster_name_or_ID>
-w <workerID_1> -w <workerID_2>
```

- g. After the worker nodes are back in a healthy state, verify that the global pull secret is updated on a worker node.

- i. Start a debugging pod to log in to a worker node. Use the **Private IP** that you retrieved earlier for the `<node_name>`.

```
oc debug node/<node_name>
```

- ii. Change the root directory to the host so that you can view files on the worker node.

```
chroot /host
```

- iii. Verify that the Docker configuration file has the registry credentials that match the global pull secret that you set.

```
vi /.docker/config.json
```

Parent topic:

→ [Prerequisites](#)